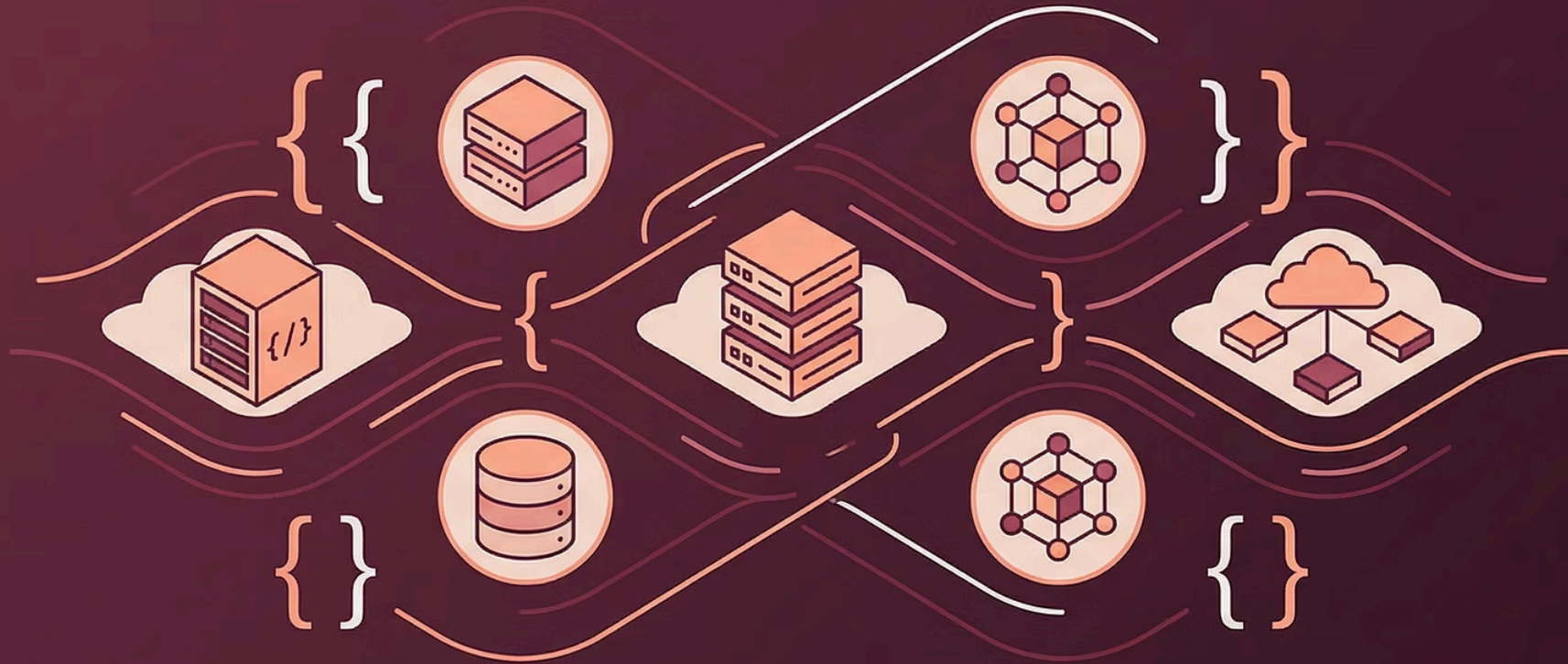


40px

Terraform Fundamentals: Building Infrastructure as Code

Master HCL syntax, providers, and the Terraform ecosystem

60px



60px

40px

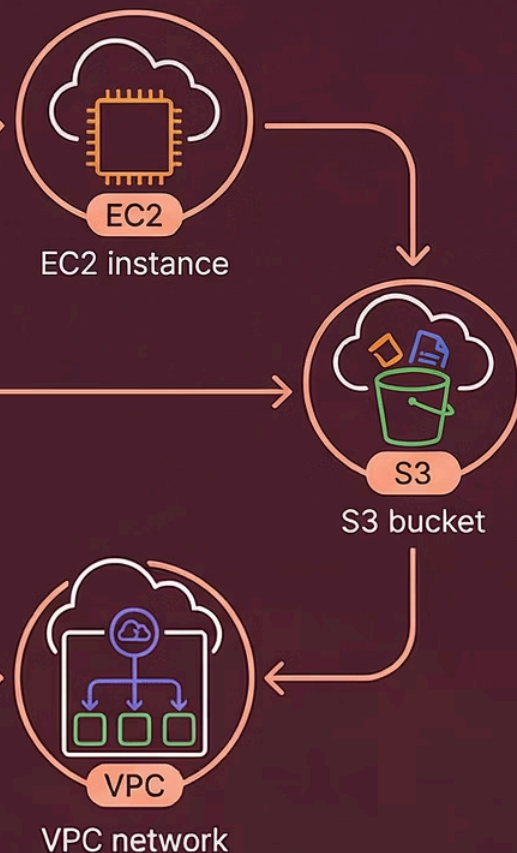


HCL Syntax: The Language of Infrastructure

HashiCorp Configuration Language uses blocks, arguments, and expressions to define infrastructure declaratively.

Each block type serves a specific purpose: resource blocks create infrastructure, data blocks fetch information, and provider blocks configure APIs.

```
1 resource "aws_instance" "web_server" {  
2   ami           = "ami-0c55b159cbfafa1f0"  
3   instance_type = "t2.micro"  
4   tags = {  
5     Name = "WebServer"  
6   }  
7 }  
8  
9 resource "aws_s3_bucket" "data_bucket" {  
10  bucket = "my-data-bucket-12345"  
11  acl    = "private"  
12 }  
13  
14 resource "aws_vpc" "main_vpc" {  
15  cidr_block = "10.0.0.0/16"  
16 }
```



Blocks: The Building Foundation

Resource blocks are the heart of Terraform - they define the infrastructure components you want to create.

```
1 provider "aws" {  
2   region = "us-west-2"  
3 }  
4  
5 resource "aws_instance" "web" {  
6   ami           = "ami-0c55b159cbfafa1f0"  
7   instance_type = "t2.micro"  
8 }  
9  
10 data "aws_ami" "ubuntu" {  
11   most_recent = true  
12   owners      = ["099720109477"] # Canonical  
13 }
```



Provider



Resource

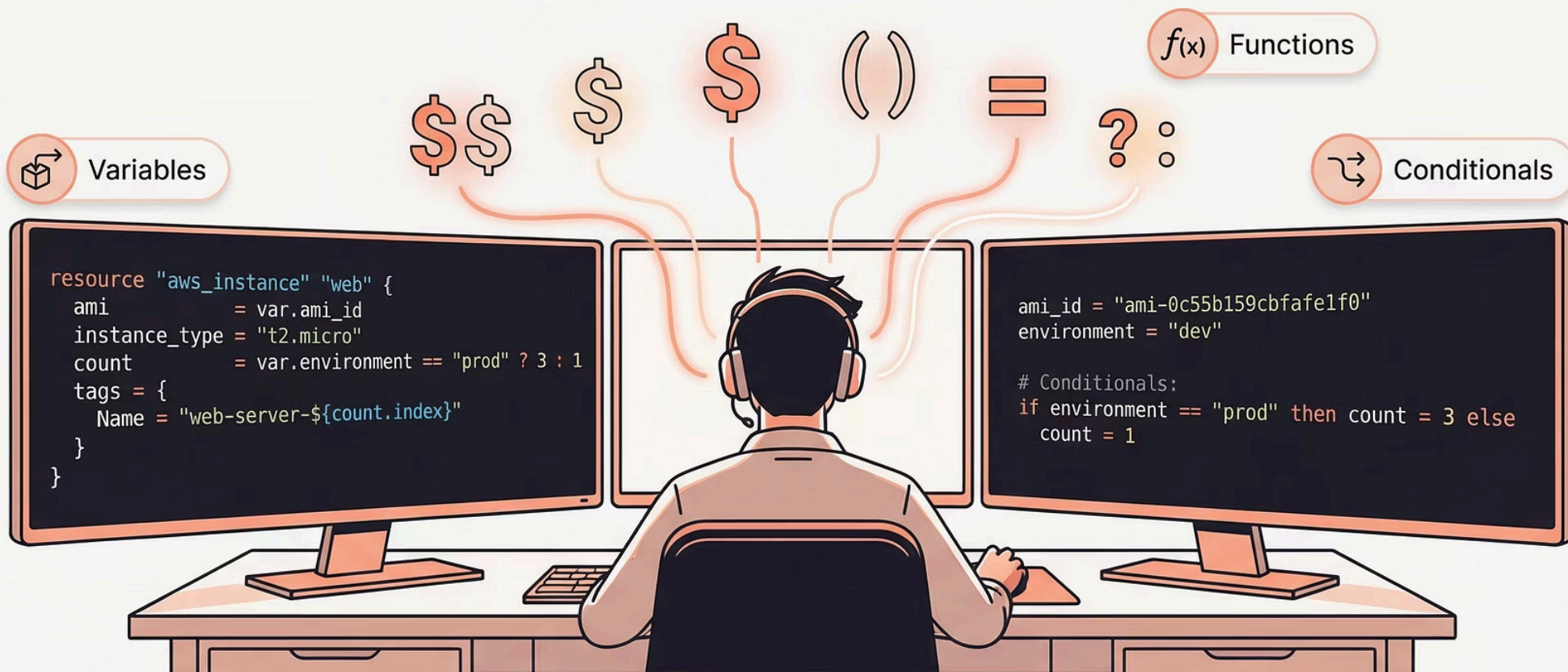


Data

Data blocks retrieve information about existing infrastructure that you can reference in your configuration.

Arguments and Expressions

Arguments define the properties of your infrastructure resources, while expressions allow dynamic values and references.

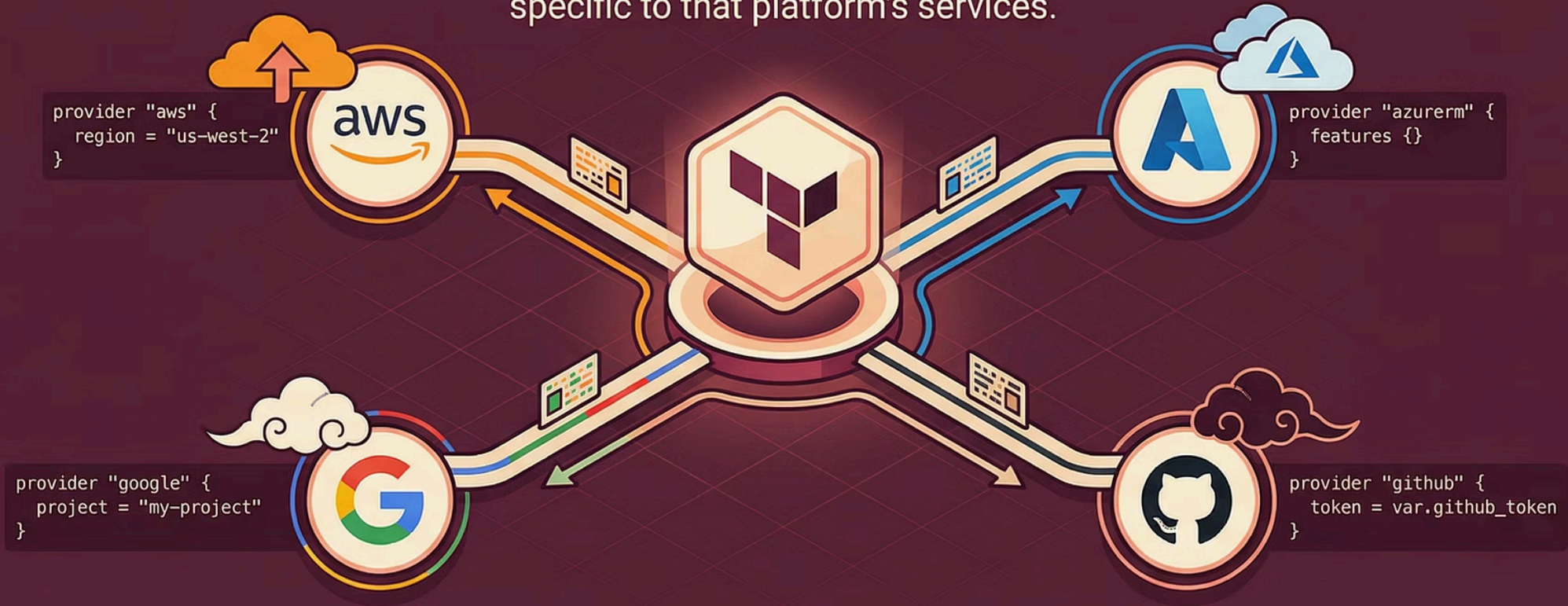


Variables, functions, and conditionals make your Terraform configurations flexible and reusable across environments.

Providers: Your Cloud Gateway

Providers are plugins that enable Terraform to interact with cloud platforms, SaaS providers, and other APIs.

Each provider translates HCL configuration into API calls specific to that platform's services.

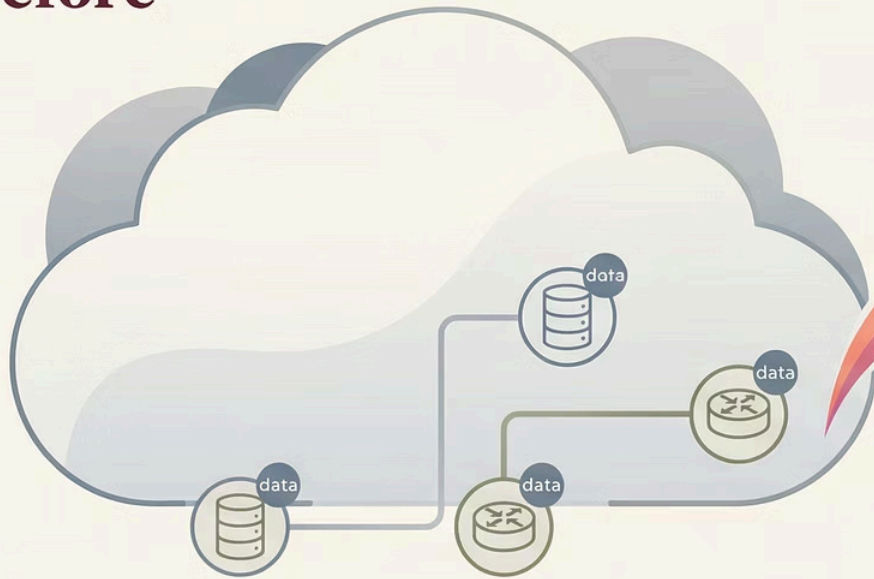


Resources vs Data Sources

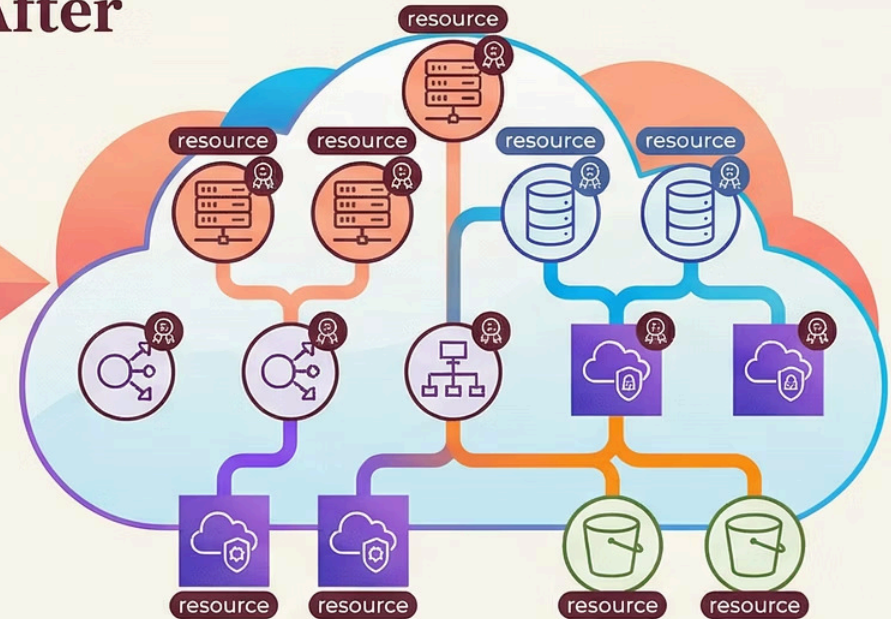
Resources create and manage infrastructure components, while data sources fetch read-only information about existing resources.

This distinction allows you to reference existing infrastructure in your new configurations safely.

Before



After



Terraform Registry: Your Resource Library

The Terraform Registry contains thousands of providers and modules contributed by HashiCorp, cloud vendors, and the community.

Browse, search, and integrate proven infrastructure patterns directly into your configurations.

The image shows a screenshot of the Terraform Registry website (registry.terraform.io) with several annotations. At the top right, two circular callouts indicate '3,000+ Providers' and '10,000+ Modules'. The main content area is divided into three sections: a left sidebar with a search bar and filters, a central 'Featured providers' section, and a right 'Module documentation preview' section. The 'Featured providers' section displays four cards for AWS, Azure, Google Cloud, and Kubernetes, each showing download counts, ratings, and versions. The 'Module documentation preview' section shows an example Terraform module for a VPC. Surrounding the website screenshot are several circular icons representing community members, a handshake icon labeled 'Community Contributed', and dimension lines indicating 40px and 60px measurements.

40px

Terraform Registry: Your Resource Library

The Terraform Registry contains thousands of providers and modules contributed by HashiCorp, cloud vendors, and the community.

Browse, search, and integrate proven infrastructure patterns directly into your configurations.

3,000+ Providers

10,000+ Modules

60px

40px

60px

Community Contributed

registry.terraform.io

Search

Providers

Modules

Categories

- ☒ Official
- ☐ Partner
- ☐ Community
- ☐ Verified

Featured providers

Provider	Downloads	Rating	Version
aws	500M+	★★★★★ 4.9	v5.0.1
Azure	450M+	★★★★★ 4.8	v4.2.0
Google Cloud	300M+	★★★★★ 4.7	v5.5.3
Kubernetes	250M+	★★★★★ 4.8	v2.1.0

Module documentation preview

```
# Example Terraform Module
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "5.0.0"

  name = "my-vpc"
  cidr = "10.0.0.0/16"

  azs = ["us-east-1a", "us-east-1b"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24"]
  public_subnets = ["10.0.101.0/24", "10.0.102.0/24"]

  enable_nat_gateway = true
  tags = {
    Terraform = "true"
    Environment = "dev"
  }
}
```